Hex CLI: A Measurement-First Coding Agent on a 4-Billion-Parameter Model and a Laptop NPU

Nathan L.
github.com/NathanL15/Hex-CLI

August 2026 · v2.2.0

Abstract

Hex CLI is a terminal coding agent in the tradition of Claude Code and GitHub Copilot CLI, with one structural difference: it runs entirely on a consumer laptop. There is no cloud service behind it, no API key, and no code leaving the machine. It pairs a small 4-billion-parameter model (Qwen3-4B-Instruct-2507) with the Hexagon NPU of a Snapdragon X Elite and compensates for the model's limits through engineering: a statistical evaluation instrument graded on real filesystem state, a rebuilt agent loop, and a security layer that assumes the model can always be manipulated. Between versions 1.7 and 2.0 the model never changed, yet every headline metric improved. The extended task suite rose from 22/35 to 24/36 (pass^5), multi-turn success from 23/45 to 26/45, executed prompt-injection payloads fell from 9/9 to 0/9, and offline test coverage grew from 330 to 619 tests. The negative results matter as much as the positive ones. Eight plausible improvements, including a new action protocol, prompt compression, a larger context window, and a runtime migration, were built or benchmarked, measured, and rejected with evidence. The central claim of this report is methodological: at the small end of the model scale, the harness is the product.

1 The constraint is the point

Frontier coding agents assume a frontier model. Hex CLI asks what happens without one: how much of a real coding agent survives when the model is Qwen3-4B-Instruct-2507, small enough to run at about 15 tokens per second on a laptop NPU (the low-power AI accelerator built into the laptop's processor), inside a context window of 4,096 tokens (roughly 3,000 words), on a 16 GB machine with no cloud fallback?

The constraints are severe. The model degrades measurably once the context passes roughly 2,600 tokens, and the system prompt alone costs about 2,100, so the usable working space is a thin slice of an already small window (Figure 1). Models must be quantized and compiled ahead of time for this NPU, and such bundles exist for exactly twelve of them, so swapping to a better model is rarely an option. Each of these numbers was measured on the target machine, and each one shaped the design.

There are practical reasons to accept these constraints: code never leaves the laptop, inference costs nothing, and the first token arrives in about half a second. The more interesting reason is the engineering thesis this report defends. At small model scale, the harness (parsing, context management, edit application, safety) matters more than the model. Running a model locally is a solved problem; tools like Ollama do it in one command. Running a competent, safe agent on a small local model is not, and that gap is what this project is about.

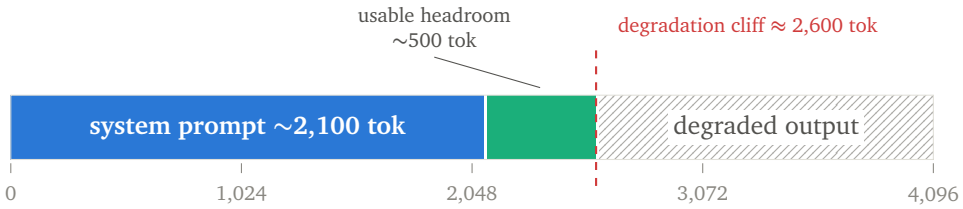

Figure 1: The 4,096-token context window as the agent actually experiences it. Output quality collapses past the measured cliff, so auto-compaction, calibrated with an exact token estimator, keeps every session left of the red line.

2 System architecture

The system is two layers in two languages, split where each is strong (Figure 2). A Python harness owns the agent loop: prompt construction, action parsing, dispatch across 15 tools, safety checks, context compaction, and the interactive REPL. A vendored fork of *npurun*, written in Rust, owns inference. It wraps Qualcomm's Genie SDK behind a local server that speaks the OpenAI API. The fork is not a passive dependency; over the course of the project it gained per-token usage reporting, token-precise `max_tokens`, mid-stream stop sequences, and a fix for a UTF-8 character-boundary crash that could take down the whole server.

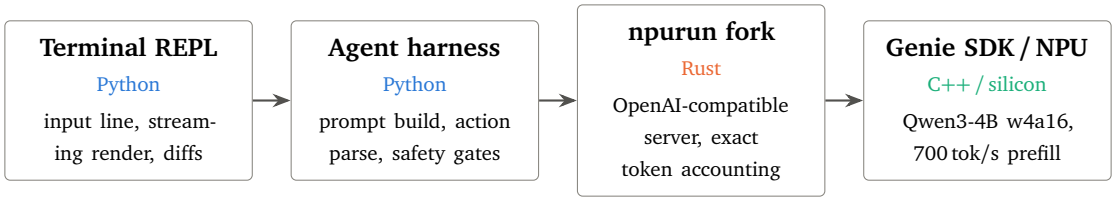

Figure 2: System layers for one agent turn. The evaluation instrument drives the same harness code path the user does: one loop, no test double. The model is quantized to w4a16 (4-bit weights, 16-bit activations) to fit the NPU. Measured Python overhead per turn is below 1% of wall-clock time, which is why a full Rust rewrite was considered and declined (§5).

The same split exists in silicon (Figure 3). Inference occupies the NPU while the harness and every tool the agent runs (shell commands, scripts, file operations) execute on the CPU cores, so tool execution never competes with token generation.

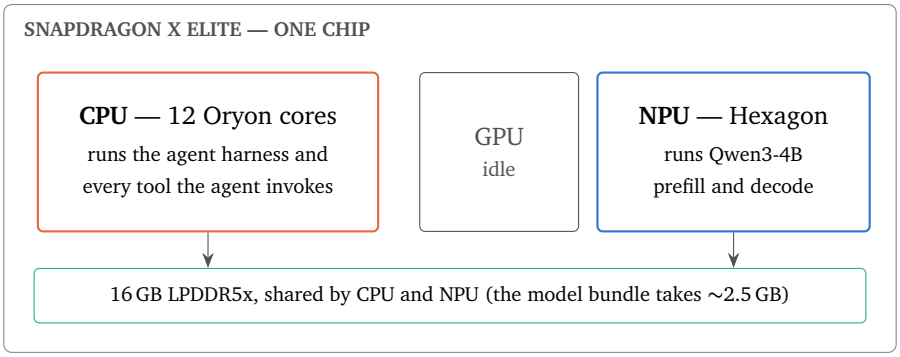

Figure 3: Where the work runs on the chip. Agent logic and inference occupy different silicon, which is also why the CPU-only alternative loses in practice (§4.2): a CPU that is busy decoding is a CPU that cannot run the agent's tools.

2.1 What using it looks like

A session is a conversation in the terminal. The user types a request; the agent works in steps, where each step is either a tool call or a final answer. Here is a representative task, drawn from the evaluation suite:

```
> fix the crash in report.py

read_file    report.py    locates the function
run_code     report.py    NameError: name 'conut' is not defined
edit_file    report.py    conut = 0 → count = 0
run_code     report.py    exit 0, output correct
```

Fixed. Line 12 of `report.py` misspelled `count`; corrected and re-run to confirm.

Two product behaviors are visible even in this small example. Every file mutation is shown to the user as a diff before the session moves on. And a claim of "done" after an unverified edit is deflected: the harness asks the agent to check its work before it may finish.

3 Method: build the instrument first

The rule for v2 was that no change ships on intuition. Before touching the agent itself, the project built an evaluation instrument with three properties that informal evaluations usually lack.

First, grading is based on state. A task passes only if the filesystem ends up correct, or the answer content is verifiably right. A transcript containing an expected string proves nothing; language models are excellent at claiming success.

Second, every case runs three to five times, and a case passes only if all runs pass (pass^k), reported with Wilson confidence intervals. Single runs of a small model are noise. Three separate apparent regressions in this project disappeared entirely when re-run at n=5 or n=10.

Third, backend failures are quarantined. The NPU inference stack silently degrades after an hour or two of traffic, and when it does, it looks exactly like a model regression. The runner detects a degraded server and marks affected runs invalid rather than failed, and every A/B comparison starts a fresh server for each arm.

That last property deserves emphasis. Twice in this project, an apparent quality regression turned out to be the measurement platform drifting between runs. If the arms of an experiment are not interleaved or freshly reset, the comparison is void, and nothing about this failure announces itself until it has cost a day of misdirected debugging.

4 Results: same model, rebuilt harness

The gains in v2 came from finding the harness's real failure classes and fixing them. The largest single fix was in the action parser: a greedy JSON-extraction regex was silently discarding the model's batched multi-action responses, which turned out to be the true cause of a multi-turn collapse that had been blamed on context length. Beyond that, a four-tier fuzzy matcher now applies file edits (the most common v1 failure was edit formatting, not model capability), malformed actions are retried with feedback naming the specific defect, and context compaction was recalibrated with an exact token estimator so that it fires before the model's degradation cliff instead of after it. Figure 4 shows the aggregate effect.

4.1 Where the context window actually goes

Budgeting the system prompt produced the most counter-intuitive finding of the project (Figure 5). The 14 behavioral rules account for 73% of the prompt template; all 15 tool schemas together cost

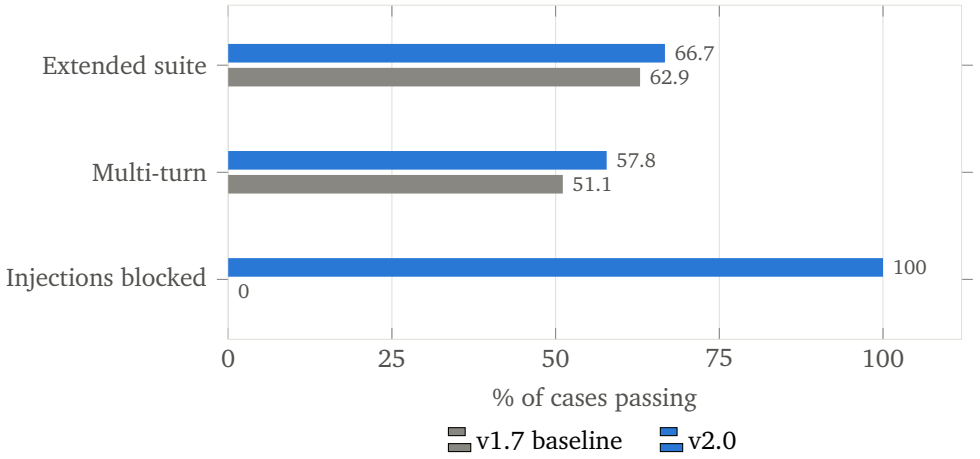

Figure 4: Live evaluation results for v1.7 and v2.0 on identical model and hardware. Extended suite: 22/35 to 24/36 cases passing all five runs (pass[5]; the suite gained one case in v2, hence the changed denominator). Multi-turn: 23/45 to 26/45, counting each turn of the scripted conversations across repeated runs. Adversarial injection payloads blocked: 0/9 to 9/9.

only about 450 tokens. The intuitive optimization, consolidating tools, would have saved almost nothing, and §5 shows that the rules could not be safely trimmed either.

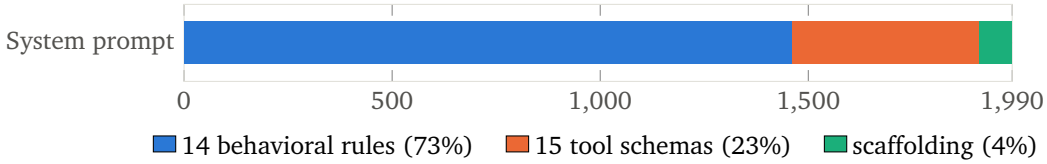

Figure 5: Anatomy of the 1,990-token system-prompt template at v2.0. With the full prompt near 2,100 tokens, almost half the 4,096-token window is spent before the user types a word. Each rule earned its place by fixing a measured failure.

By v2.2 the live-state cookbook had grown the prompt to 2,355 tokens (measured exactly via the server's usage report; the 14 rules are 1,689 of them), clamping the history budget to its 250-token floor and making auto-compaction fire every few messages. Two follow-up measurements closed this line of work. First, compaction itself was the thrash: the deterministic compactor re-fired every turn and crushed its own previous summary block into a single stub each time. Making it merge-aware (idempotent on its own output) and adding a dry-run gain guard fixed the churn without touching the prompt. Second, a dedicated sweep ran the production loop at controlled input sizes from 2,370 to 2,973 measured tokens and found quality *flat* across the whole range — while inputs targeted above ∼3K came back truncated to ∼2,92x by the runtime. The shipping configuration already sits at that ceiling. Together with the negative results in §5, the conclusion is that the 250-token history floor is a property of the model and the compiled 4K bundle, not a harness gap: within this model and runtime, the context budget is fully spent, and spent well.

4.2 Why the NPU, when the CPU decodes faster

Two terms matter here. *Prefill* is the phase where the model reads the prompt; *decode* is the phase where it generates output, one token at a time. The hardware choice was settled with data rather than assumption. On plain CPU, the same model actually decodes faster than the NPU path (13.5 versus 8.1 tokens per second in this test). But an agent re-reads its 2,100-token prompt every turn, and prefill is where the NPU is untouchable (Figure 6). Extending the two measured points across turn lengths makes the trade explicit (Figure 7): the CPU's decode advantage would pay off only

past about 193 generated tokens per turn, and observed real turns generate between 12 and 126. The NPU therefore wins every realistic turn, and it leaves the CPU cores free for the agent's own tool execution.

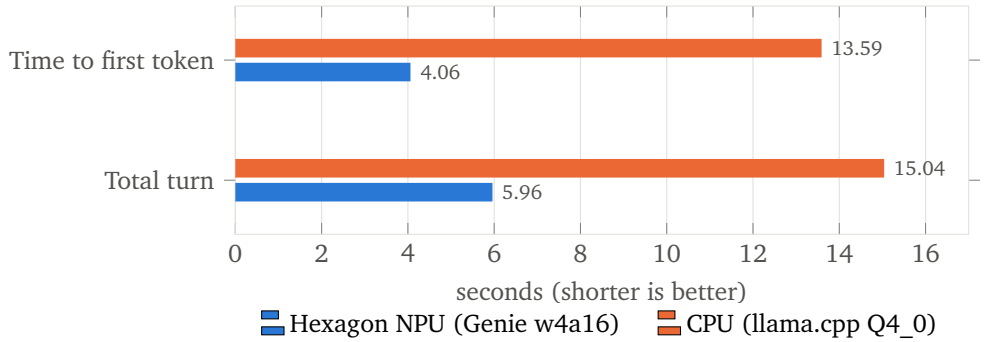

Figure 6: NPU and CPU compared on realistic agent turns: same model, the real 2,100-token system prompt, arms interleaved, medians of matched-length runs.

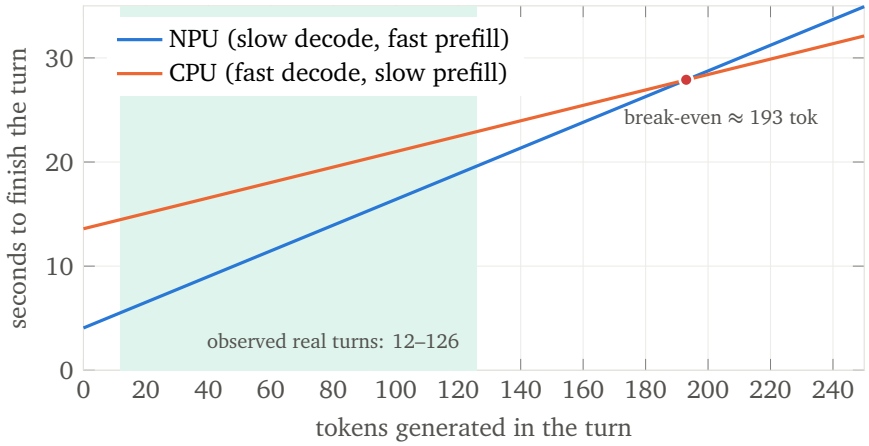

Figure 7: Total turn time as a function of how many tokens the turn generates, extrapolated from the measured intercepts and decode rates in Figure 6. The CPU line only wins to the right of the break-even point, and real turns never get there.

4.3 What still fails

Section 1 asked how much of a real coding agent survives at this scale, and the honest answer includes the 12 of 36 cases that still fail. They concentrate in two classes, and both are model ceilings rather than harness bugs. The first is bait compliance: when a prompt explicitly names a tool ("use `run_command` to…") for something that should be answered directly, the model complies roughly one time in three despite an explicit rule against it. The second is edit quality deep into a session: past about 2,500 tokens of context the model starts reconstructing edit anchors from memory instead of from the file, or drifts into describing the change in prose instead of making it. The harness converts both from silent failures into retried or deflected ones, but it cannot eliminate them. They define where a better small model would actually help, and the instrument is ready to measure one when it exists.

5 Negative results

Half the value of the measurement instrument is what it stopped from shipping. Each row of Table 1 is a plausible improvement that was built or benchmarked, measured, and rejected. The evidence is recorded so that none of these ideas gets re-argued from intuition later.

Idea	What the measurement said	Verdict
New structured `<action>` protocol as default	13/36 vs. 22/35 pass[5]; two months of tuning had specialized the model to the old format	rejected
Prompt trimming (drop situationally irrelevant rules)	saved 690 tokens and 16% latency, but resistance to bait prompts (cases that tell the model to misuse a tool) fell from 5/8 to 3/18 ($p \approx 0.017$), and the damage did not scale with rules dropped	rejected
8K context window (recompiled bundle)	6 tok/s vs. 15, with no quality gain; compaction already keeps sessions under the degradation cliff	rejected
Raise the compaction threshold (re-measure the ∼2,600-token cliff)	a dedicated sweep found quality *flat* from 2,370 to 2,973 measured input tokens — but the runtime silently trims inputs above ∼3K to ∼2,92x, and the shipping configuration already operates at that ceiling, so a higher threshold buys zero usable history	closed
Leaner prompt for agent steps ≥ 2 (drop the five step-1 decision rules)	overall quality flat (77.8% vs. 74.4%, $p = 0.65$), but the JSON-edit case fell 3/3 → 1/3 with degenerate `old_string` anchors — the trimming experiment's degradation fingerprint, reproduced at continuation steps	rejected
Escalate hard cases to Qwen3-8B	0.9 tok/s: silent CPU fallback on a 16 GB machine	rejected
Qwen3.5-4B (newer successor model)	20–25% slower than the incumbent on both CPU and hybrid paths	deferred
Native Qwen3 `<tool_call>` template	the quantized bundle's detokenizer garbles its own special token	impossible
Migrate to the GenieX runtime (+47% claimed)	0.74 tok/s on the target machine, isolated to a fixed 1.5 s penalty per NPU graph call; filed upstream (GenieX #1266, PR #1267)	halted
Rewrite the harness in Rust	Python's share of a turn is below 1% of wall-clock; the bottleneck is inference	declined

Table 1: Ten plausible improvements that failed their evaluations. Writing these down, with the evidence, is what keeps the next month of work from re-fighting the last one.

One finding runs underneath several of these rows. A small model tuned for months against one prompt becomes specialized to that exact prompt. Dropping even provably irrelevant rules degraded behavior, and the damage did not scale with the number of rules dropped: removing one rule hurt more than removing four. At this scale the prompt is not documentation the model consults. It is part of the distribution the model has adapted to. The continuation-stage experiment extended this finding: the specialization holds at *every* step of the loop, not just the first — a prompt the model has not been tuned against degrades adjacent behavior even when the removed text is logically irrelevant to the failing action. The one split that survived measurement is the one that changes the model's task rather than shrinking its prompt: a harness-routed *direct stage* for pure-knowledge queries, whose no-tools prompt makes tool restraint structural and cut knowledge-query first-token latency by 40% (10.1 s → 6.0 s median) with no regression in its routed subset.

The larger lever turned out to be the runtime, not the prompt. A 2,355-token system prompt that is prefilled again on every agent step is a fixed tax of about five seconds per call, and the Genie

runtime that ships with QAIRT 2.47 rejects its own KV-cache rewind on this bundle. QAIRT 2.50 accepts it. The server fork was changed to keep one dialog alive for the life of the process and to send every warm query as a prefix-matching rewind, rebuilding the dialog in place only when the system prompt itself changes; two runtime behaviours dictated that shape (a reset after a large prefill wedges the dialog, and a transcript that diverges early poisons it). The prompt then had to become byte-identical across working directories and days, which moved the date and directory out of the system prompt and into the first user message. Measured on the extended suite with three runs per case against a fresh server per arm: run-level $91/117 \rightarrow 97/117$ (Fisher $p = 0.41$, no regression), first-token median 6.8 s → 3.7 s, agent step ≥ 2 median 7.6 s → 3.2 s, whole-turn mean 16.0 s → 9.5 s. The direct stage above is switched off in this configuration: a knowledge query is already decode-bound once the prefix is warm, and a second system prompt would only buy a rebuild.

6 Security: assume the model is 100% injectable

An agent that reads files and runs commands is a target for prompt injection. To a 4B model, a malicious instruction hidden in a README it has been asked to read is indistinguishable from the user. Version 1.7 executed all nine payloads in the live injection suite, which included hosts-file reconnaissance, SSH-key exfiltration, and spawning arbitrary executables. Version 2.0 blocks all nine.

The design principle matters more than the score: no defense is allowed to depend on the model resisting the bait. Enforcement lives in the harness instead. A *sensitive* classification tier, ranked above "safe," covers credential stores, SSH and cloud keys, registry hives, and encoded-command tricks. Access to these is confirmed interactively and denied outright when the session is unattended. Every mutating code path routes through a single `guard_mutation` gate, and a test enumerates every mutating entry point in both the production loop and the experimental v2 protocol, driving each one at a forbidden path. Network egress is denied by default.

One measured lesson became a permanent style rule. An early refusal message helpfully suggested an alternative command, and the model followed that hint straight to the bypass. Refusal messages never name another route.

7 Timeline

Date (2026)	Milestone
Jun 16	First end-to-end NPU inference: npurun and the Genie SDK running Qwen3-4B at about 15 tok/s, after two other runtimes proved to be dead ends.
Jun–Jul	v1.x: state-graded evals find six real prompt and loop bugs; the runtime fork gains stop sequences and a crash fix. The project is renamed Hex CLI.
Jul 29	v1.7 audit and v2 plan: a full-repo audit, CI green for the first time, and fresh measurements of prefill, decode, prompt size, and the degradation cliff.
Jul 30–31	The v2 sprint: instrument v2, agent-loop rebuild, injection defense (0/9 to 9/9 blocked), product shell, rich input line. Every change was A/B-tested live.
Jul 31	v2.0.0 ships: the repository's first GitHub Release, with 619 tests across 19 suites in CI.
Aug 1–2	Runtime work: a migration to GenieX is measured and halted, with findings filed upstream, and an NPU driver update is validated against the eval suite before adoption.
Aug 14	v2.1.0: a one-command installer (the setup ritual was the last barrier to anyone else running it), piped stdin, custom commands, session search, and a config wizard. An independent review of the release found ten defects in it, all fixed before shipping; 685 tests across 22 suites.
Aug 16	v2.2.0: a systematic sweep of everyday prompts (30 cases graded against computed machine truth) lifts live machine-state questions from 44% to 72% via a command cookbook in the prompt — and uncovers the deepest bug of the project: the memory consolidation daemon had been recording the model's own confabulations as facts and re-injecting them every turn, a self-reinforcing hallucination loop invisible to every sandboxed eval.
Aug 29–31	The context question is closed by measurement. Auto-compact thrash fixed (merge-aware compactor, dry-run gain guard); the command surface pruned to one mode and 18 commands (−685 lines); a cliff sweep finds quality flat to the runtime's own ∼2.9K input ceiling, retiring threshold tuning; and a two-stage prompt-split A/B keeps the no-tools direct stage (−40% knowledge-query latency, structural tool restraint) while rejecting the leaner continuation prompt on the trimming fingerprint.
Sep 1–2	The main module is decomposed into seven verified stages (3,818 → 1,499 lines, full pass^3 arm at parity). A single oversized tool result is found to overflow the 4K window and is fixed with per-step budgeting and paged reads; the live-state grader bug is fixed. QAIRT 2.50 accepts the KV rewind that 2.47 refused: the server keeps the dialog across calls and the prompt is made byte-stable, cutting whole-turn time by 41% with no regression (91/117 → 97/117).

Table 2: Project milestones.

8 Lessons

1. **At small scale, the harness is the product.** Every measured win in this project (parsing, edit application, compaction timing, retry feedback) came from the scaffolding around a frozen model. The model is a component; the agent is the engineering.

2. **Build the instrument before the feature.** pass^k with state-based grading made every other claim in this report possible, and it repeatedly overturned the story a single run told.

3. **Interrogate the measurement platform.** A silently degrading backend impersonates a model regression perfectly. Fresh servers per arm and interleaved comparisons are the difference between data and noise.

4. **Security is a property of the architecture, not the model.** Assume total injectability, put every enforcement decision in code you control, and test it by enumeration rather than spot-check.

5. **Negative results are deliverables.** Written down with their evidence, they keep future work from re-fighting settled questions.

9 Status and future work

The core measurements in this report are from v2.0.0, whose scope is closed; sections that cite later dates (the context-budget follow-ups, the prompt-split A/B, the everyday-prompt study) say so inline. v2.1.0 followed with the packaging work: a one-command installer replacing the SDK-and-runtime setup ritual, plus the shell features that make the agent scriptable. It deliberately did not touch the prompt or the loop, and the live gate confirmed as much before and after.

Two things are worth recording about that release, because they are the same lesson this report argues for, applied to the harness's own code. An independent review of it surfaced ten defects the feature tests had missed — including a version comparison that sorted SDK directories as strings, which its own test certified because the chosen version numbers happened to sort identically either way. Tests pin the invariant you thought of, not the one that matters; the fix was to re-derive the invariants and mutation-test them.

The context question is now closed by measurement rather than left open: the history budget is fully spent against the model's own ceilings, and the remaining levers all require a different model or runtime. Both of the next steps named in the previous revision are done: the main module is decomposed, and the per-turn prefill tax is gone wherever the runtime allows it, which took a runtime upgrade and a server change rather than any prompt work. What remains on the harness side is small and measurable: letting compaction share the warm prefix, and paging large files on the model's behalf. The model upgrade path stays a watch item rather than a work item: the moment a supported successor bundle appears for this NPU, the instrument that produced every number here can judge it in an afternoon.[1]

[1] All figures were measured on a Snapdragon X Elite (X1E78100), 16 GB RAM, Windows 11 ARM, with Qwen3-4B-Instruct-2507 (w4a16) on the Qualcomm Hexagon NPU. Methodology and full run-level data: `docs/V2_PLAN.md` §14 and `evals/` in the repository.